

HelixCluster

HelixCluster

Un système d'exploitation distribué pour le calcul AI — des GPU en datacenter aux appareils mobiles en périphérie, sous un seul plan de contrôle.

Résumé

Helix Cluster OS est un système d'exploitation distribué de nouvelle génération qui orchestre le calcul sur des nœuds hétérogènes — des GPU en datacenter aux nano-ordinateurs et appareils mobiles en périphérie — unifiant la planification HPC, l'orchestration de conteneurs, l'inférence AI/ML, l'exploitation multi-clusters fédérés et les sessions multi-locataires sécurisées sous un seul plan de contrôle.

Description courte

Un système d'exploitation distribué basé sur Go / cluster de calcul partageant des ressources GPU. Il unifie la planification HPC (un ordonnanceur à deux niveaux de type Omega), l'orchestration de conteneurs, le routage d'inférence AI, la fédération et les sessions multi-locataires sécurisées sur des nœuds hétérogènes, coordonnés par un protocole de gossip SWIM et un consensus Raft, avec un chiffrement de bout en bout post-quantique.

Description longue

Helix Cluster OS orchestre les charges de calcul sur des matériels radicalement hétérogènes — GPU en datacenter, nano-ordinateurs en périphérie, voire appareils mobiles — sous un seul plan de contrôle, traitant un rack d'A100 et une poignée de nano-ordinateurs comme un seul tissu adressable plutôt qu'une douzaine d'îlots incompatibles. Il s'agit d'un espace de travail Go (un monorepo avec des sous-modules git) implémentant une pile à sept couches, du substrat matériel L0 jusqu'à la fédération L7 et

l'observabilité, coordonné par quatorze microservices de plan de contrôle. L'appartenance des nœuds est suivie via un protocole de gossip SWIM et de découverte, permettant au tissu de s'auto-réparer à mesure que les nœuds rejoignent ou quittent le réseau ; l'état fortement cohérent repose sur un consensus Raft, organisé en groupes Raft par shard avec des lectures locales au détenteur du bail pour la rapidité et un mécanisme STONITH pour garantir qu'un nœud partitionné ne puisse corrompre l'état partagé. Le placement des charges de travail passe par un ordonnanceur à deux niveaux de type Omega — concurrence optimiste, appariement ClassAd, ordonnancement en gang, préemption par multiplicateur de valeur et placement basé sur des contraintes — puis va plus loin qu'un ordonnanceur HPC classique : routage tenant compte de l'empreinte carbone et du coût/TCO, mise à l'échelle automatique vers le cloud en cas de pic, et des adaptateurs de marché (Akash, io.net, RunPod, AWS Spot, Chutes) permettant à une tâche de déborder sur des capacités louées lorsque les ressources locales viennent à manquer.

Les utilisateurs finaux ne voient rien de cette machinerie ; ils interagissent via un modèle de session épuré (allocations de calcul), un terminal interactif WebSocket/PTY, un routage interne d'inférence AI et des lectures de l'utilisation des pools. La sécurité est une couche native et non un ajout : identité SPIFFE, attestation des appareils (défi/réponse, preuve de travail GPU, scellement), une porte KYC de contrôle des exportations, et un transport chiffré de bout en bout post-quantique construit sur un échange de clés hybride X25519 + ML-KEM-768 avec protection des enregistrements AEAD et rejet des rejeux — conçu pour que le trafic capturé aujourd'hui reste confidentiel même face à un adversaire quantique de demain. La justesse n'est pas affirmée, elle est *démontrée* : des tests de simulation déterministe (exécutions avec graines à la FoundationDB, injection de fautes, simulation de réseau, rejeu octet par octet et un vérificateur de linéarisabilité Porcupine) reproduisent les défaillances distribuées à la demande, et des tests de mutation appariés obligatoires prouvent que les tests de garde mordent réellement. L'architecture et la documentation sont maintenues en cohérence par des linters mécaniques qui font échouer la compilation dès que la réalité et la documentation divergent.

Contenu

Pourquoi nous l'avons conçu

Pour exécuter AI et des charges de travail HPC sur des niveaux matériels radicalement différents sans avoir à assembler des ordonnanceurs, des orchestrateurs et des piles d'inférence distincts — et pour le faire avec une garantie technique : chaque fonctionnalité livrée doit prouver un *comportement réel de l'utilisateur final* (jamais de tests en environnement simulé sur des

stubs) et chaque capacité spécifique à un système d'exploitation doit s'appuyer sur une fonction native réelle par plateforme (pas de simulations Linux-only). Le problème fondateur, cité dans la gouvernance du dépôt, est ce mode d'échec où « les tests passent, mais la fonctionnalité ne fonctionne pas réellement », que le projet a été explicitement conçu pour éliminer.

Pourquoi c'est révolutionnaire

Il fusionne cinq éléments qui forment habituellement cinq piles distinctes — l'ordonnancement HPC, l'orchestration de conteneurs, l'inférence AI, la fédération multi-clusters et les sessions multi-locataires sécurisées — en un seul plan de contrôle s'étendant des GPU de datacenter jusqu'aux appareils portables en périphérie. Et il le fait avec un niveau d'exigence généralement réservé aux infrastructures spécialisées : une correction de niveau méthodes formelles (spécifications TLA+, simulation déterministe, vérification de linéarisabilité) et un transport confidentiel post-quantique, des garanties que la plupart des orchestrateurs n'osent même pas tenter. Au-delà de la différenciation technique, le placement conscient des coûts et de l'empreinte carbone, couplé à l'éclatement vers les marchés cloud, en fait aussi un levier *économique* — l'ordonnanceur peut automatiquement rechercher des capacités moins chères, plus vertes ou inutilisées, permettant ainsi à une même charge de travail de coûter moins cher et d'émettre moins, sans que quiconque ait à réécrire un job.

Ce qui est innovant

- **Tests de simulation déterministe (DST)** — un simulateur ensemencé, entièrement reproductible, qui injecte des pannes, des décalages d'horloge et des partitions réseau, les rejoue octet par octet, et soumet le résultat à un vérificateur de linéarisabilité Porcupine. Ainsi, un Heisenbug détecté une fois peut être reproduit à la demande, indéfiniment.
- **Ordonnanceur à deux niveaux de type Omega** — placement par concurrence optimiste avec appariement ClassAd, ordonnancement par gang et préemption par multiplicateur de valeur, une conception à état partagé permettant à plusieurs ordonnanceurs de s'engager sur un même cluster sans goulot d'étranglement central.
- **Câblage E2EE post-quantique / inférence confidentielle** — un échange de clés hybride X25519 + ML-KEM-768 avec liaison par paire de clés de réponse par requête et AEAD avec rejet des rejeux (les primitives cryptographiques sont réelles et testées ; le tour complet confidentiel multi-nœuds reste explicitement PLANIFIÉ/verrouillé).
- **Confiance basée sur l'attestation** — les nœuds doivent *prouver* ce qu'ils sont : identité SPIFFE, preuve de travail GPU,

scellement de périphérique, une porte KYC de contrôle des exportations et génération de documents conformes au règlement AI de l'UE. La confiance s'acquiert par des preuves, plutôt que d'être supposée en fonction de la position dans le réseau.

- **Orchestration consciente des coûts et de l'empreinte carbone** — modélisation du TCO, placement tenant compte de l'empreinte carbone, éclatement vers le cloud, une réserve de basculement N+K et des adaptateurs pour les marchés cloud, faisant des coûts et des émissions des paramètres de premier plan pour l'ordonnancement, plutôt que des réflexions a posteriori.
- **Consensus Multi-Raft** — des groupes Raft par shard avec lectures locales au détenteur du bail pour une cohérence à faible latence, soutenus par un fencing STONITH (IPMI / EC2 / Azure / SBD) afin qu'un nœud bloqué soit définitivement retiré, plutôt que laissé à corrompre l'état.
- **Linters mécaniques anti-dérive** — archlint fait échouer la compilation dès qu'un composant documenté pointe vers un chemin de paquet inexistant, et un moteur de chaîne de documentation maintient la cohérence octet par octet entre Markdown / HTML / PDF / DOCX, empêchant ainsi la documentation de mentir silencieusement sur le code.

Contenu

Principaux défis techniques et solutions apportées

- **« PASS-bluff » (tests réussissant sur des fonctionnalités non fonctionnelles).** Le mode de défaillance que l'ensemble du projet vise à éliminer : une suite de tests au vert reposant sur des bouchons. Solution adoptée : des tests de mutation obligatoires en binôme - chaque tâche intègre un test de garde nommé qui doit *échouer* suite à une mutation indépendante du code avant que la tâche ne soit marquée comme terminée, garantissant ainsi qu'un test réussi valide bien un comportement réel plutôt qu'un simulacre.
- **Parité multiplateforme (pas de bouchons spécifiques à Linux).** Solution : une interface partagée, répartie par balises de compilation sur des fonctionnalités natives par OS - cgroup Linux / /proc / WireGuard noyau, sysctl macOS / vm_stat / IOKit / wireguard-go - puis vérifiée par un oracle OS indépendant, afin que chaque plateforme rapporte un état natif authentique plutôt qu'une fiction Linux.
- **Correction distribuée en cas de défaillances.** Solution : des tests de simulation déterministes et un vérificateur de linéarisabilité qui génèrent et rejouent des partitions, des pannes et des dérives d'horloge, soutenus par des spécifications formelles TLA+ définissant les invariants de

consensus et d'ordonnancement avant même l'écriture d'une ligne de code.

- **Dérive de la documentation et de l'architecture.** Solution : archlint, qui fait échouer la compilation en cas de référence à un paquet documenté mais inexistant, et une porte de vérification docs_chain sans échappatoire - toute dérive devient une rupture de build, et non une simple page wiki obsolète.
- **Portée honnête du travail non achevé.** Le tour de table confidentiel d'inférence multinœud est délibérément protégé par un ticket et étiqueté « *non encore validé de bout en bout* » plutôt que présenté comme livré - la même rigueur s'applique à ce qui *n'est pas encore fait* qu'à ce qui l'est.

Pile technologique

- **Go (go.mod : 1.25 / chaîne d'outils 1.26.4)** - le langage du plan de contrôle pour un espace de travail d'environ 30 modules ; choisi pour sa concurrence légère via les goroutines et ses binaires statiques, déployables à l'identique du datacenter à la périphérie.
- **Zig (0.14+) + C/C++** - utilisé là où le runtime de Go pose problème : primitives système bas niveau et noyaux GPU nécessitant un contrôle déterministe et sans allocation sur le matériel.
- **gRPC + Protocol Buffers** - chaque API inter-systèmes (api/v1/) est un contrat typé et versionné, permettant aux quatorze microservices d'évoluer sans se briser mutuellement ni recourir à des formats de sérialisation artisanaux.
- **Raft (etcd-raft) + gossip SWIM** - une séparation intentionnelle : Raft gère les états nécessitant une cohérence forte, tandis que le gossip SWIM prend en charge l'appartenance et la découverte à grande échelle, là où le consensus serait trop lourd.
- **PostgreSQL 16, Redis 7, etcd v3.5, SQLite** - le bon outil pour chaque besoin : PostgreSQL pour les données relationnelles durables, Redis pour le cache à chaud, etcd pour la coordination, et SQLite embarqué pour le registre local des éléments de travail HXC.
- **NATS 2.10 (JetStream), Kafka 4.0 (KRaft), RabbitMQ 3.13** - trois dorsales de messagerie pour trois types de trafic : NATS/JetStream pour l'événementiel interne rapide, Kafka pour les flux haute performance durables, RabbitMQ pour les sémantiques classiques de courtier.
- **Maillage WireGuard + ML-KEM-768/X25519 + AES-256-GCM/ChaCha20-Poly1305 + HKDF** - WireGuard pour un maillage léger nœud à nœud, enveloppé dans une poignée de main post-quantique hybride et des enregistrements AEAD, assurant la confidentialité du transport face aux attaques classiques et quantiques.

- **SPIFFE + JWT (HS256) + RBAC basé sur les périmètres + OPA** – une identité et une autorisation en couches : SPIFFE pour l'identité des charges de travail, JWT pour les jetons, RBAC basé sur les périmètres pour un accès grossier, et OPA pour exprimer des politiques fines sous forme de code.
- **Prometheus v2.50, Grafana 10.4, Jaeger 1.55, traçage W3C** – métriques, tableaux de bord et traces distribuées avec propagation de contexte W3C, permettant de suivre une requête à travers les services et les couches matérielles.
- **HashiCorp Vault 1.16** – les secrets et le matériel cryptographique sont maintenus hors du code et de la configuration, et délivrés sous audit.
- **Docker Compose, Kubernetes (kustomize, contextes de sécurité renforcés), Helm** – Compose pour le déploiement local et Kubernetes/Helm avec des contextes de sécurité renforcés pour les déploiements réels, une seule définition promue à travers les environnements.
- **React + TypeScript + Vite (Node 20+)** – une interface web rapide et typée pour les sessions, les terminaux et l'utilisation des pools.
- **TLA+** – spécification formelle des invariants de consensus et d'ordonnancement, permettant de prouver les propriétés les plus difficiles à tester dès la phase de conception, avant toute implémentation.

Contenu

Notes sur l'état et l'honnêteté

- **État : en développement.** Cette version est précoce (0.1.0-dev). Plusieurs fonctionnalités avancées — comme l'inférence multi-nœuds confidentielle complète en aller-retour, le règlement sur la place de marché et la population d'ordonnancement basée sur les attestations — sont explicitement marquées comme **PRÉVUES** / dépendantes de l'infrastructure dans le dépôt et **ne sont pas** présentées comme pleinement opérationnelles. Les chiffres de couverture sont auto-déclarés.
- **Licence : à déterminer.** Non clairement déclarée ; les URL HelixCluster/HelixCluster et helixcluster.io du graphique Helm sont des espaces réservés non vérifiés qui ne correspondent pas aux dépôts distants réels.
- Les projets de la pile LLM intégrés (LLMOrchestrator, LLMProvider, LLMsVerifier) sont des sous-modules découplés, et non des serveurs de modèles hébergés au sein du cluster.

Niveau de priorité : Helix-primaire (cluster LLM-infrastructure — le substrat de calcul pouvant héberger des charges de travail d'inférence et de calcul). Se classe après HelixTrack.